3장. 기본 CRUD 구현하기

본 장에서는 웹 애플리케이션 개발의 근간이 되는 **CRUD(Create, Read, Update, Delete)** 개념을 학습하고, FastAPI를 통해 메모리 기반의 할 일(Todo) 관리 API를 설계하고 구현합니다. 모든 코드는 최신 파이썬 3.10+ 스타일 가이드라인을 따릅니다.

3.1 REST API와 HTTP 메서드

REST API(Representational State Transfer API)는 웹의 기존 기술과 HTTP 프로토콜을 그대로 활용하기 위한 아키텍처 스타일입니다. HTTP 메서드는 리소스에 대해 수행할 작업의 종류를 명시합니다.

HTTP Method	CRUD 매핑	상태 코드 (성공)	역할
GET	Read (조회)	200 OK	지정한 리소스나 컬렉션을 조회합니다.
POST	Create (생성)	201 Created	새로운 리소스를 생성합니다.
PUT	Update (전체 수정)	200 OK	지정한 리소스의 전체를 교체합니다.
DELETE	Delete (삭제)	204 No Content	지정한 리소스를 삭제합니다.

3.2 GET API: 할 일 목록 조회하기

먼저 메모리에 적재된 Todo 데이터 구조를 다룰 Pydantic 데이터 모델을 정의하고 전체 목록을 조회하는 API를 작성합니다.

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

# 할 일 데이터 모델 정의
class Todo(BaseModel):
    id: int
    title: str
    content: str
    is_done: bool = False

# 임시 인메모리 데이터베이스 데이터
todo_db: list[Todo] = [
    Todo(id=1, title="FastAPI 공부", content="공식 문서 1장 읽어보기", is_done=True),
    Todo(id=2, title="파이썬 실습", content="3장 기본 CRUD 코딩 완성하기", is_done=False)
]

@app.get("/todos", response_model=list[Todo])
def get_all_todos() -> list[Todo]:
    """전체 할 일 목록을 반환합니다."""
    return todo_db
```

3.3 GET API: 단일 항목 조회 및 예외 처리 (404)

전체 목록 조회가 잘 수행된다면, 특정 ID를 가진 단일 할 일을 상세 검색하는 경로를 구성합니다. 만약 존재하지 않는 ID가 유입되면 예외 처리(HTTPException)를 명시해 줍니다.

```
from fastapi import HTTPException

@app.get("/todos/{todo_id}", response_model=Todo)
def get_todo_by_id(todo_id: int) -> Todo:
    """특정 ID를 가진 할 일 항목을 검색합니다."""
    for todo in todo_db:
        if todo.id == todo_id:
            return todo

    # 존재하지 않는 경우 404 에러와 안내 메시지 반환
    raise HTTPException(status_code=404, detail="해당 ID의 할 일을 찾을 수 없습니다.")
```

3.4 POST API: 새로운 할 일 등록하기

사용자로부터 새로운 데이터를 입력받아 등록하는 POST API를 생성합니다. 생성할 때 데이터 구성을 위한 전용 요청 스키마(TodoRequest)를 설계하는 것이 깔끔합니다.

```
class TodoRequest(BaseModel):
    title: str
    content: str
    is_done: bool = False

@app.post("/todos", response_model=Todo, status_code=201)
def create_todo(request: TodoRequest) -> Todo:
    """새로운 할 일을 추가합니다."""
    # 자동 ID 할당
    new_id = len(todo_db) + 1 if todo_db else 1
    new_todo = Todo(
        id=new_id,
        title=request.title,
        content=request.content,
        is_done=request.is_done
    )
    todo_db.append(new_todo)
    return new_todo
```

3.5 PUT & DELETE API: 수정 및 삭제

기존 할 일 정보를 수정하는 PUT 요청과 특정 항목을 리스트에서 완전히 제거하는 DELETE API를 구성하여 CRUD 세트를 완성합니다.

```python
@app.put("/todos/{todo_id}", response_model=Todo)
def update_todo(todo_id: int, request: TodoRequest) -> Todo:
    """기존 할 일의 내용을 수정합니다."""
    for todo in todo_db:
        if todo.id == todo_id:
            todo.title = request.title
            todo.content = request.content
            todo.is_done = request.is_done
            return todo

    raise HTTPException(status_code=404, detail="수정할 할 일을 찾을 수 없습니다.")

@app.delete("/todos/{todo_id}", status_code=204)
def delete_todo(todo_id: int) -> None:
    """지정된 ID의 할 일을 목록에서 삭제합니다."""
    for index, todo in enumerate(todo_db):
        if todo.id == todo_id:
            todo_db.pop(index)
            return

    raise HTTPException(status_code=404, detail="삭제할 할 일을 찾을 수 없습니다.")
```

💡 **상태코드 204 (No Content) 안내**

DELETE와 같이 처리가 성공적으로 완수되었고, 클라이언트에 추가로 반환할 결과 데이터(본문)가 아무것도 없을 때 반환하는 공식적인 HTTP 규격 코드입니다. FastAPI에서는 `status_code=204`와 함께 `None` 혹은 빈 값을 반환합니다.